BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

B.Sc. Computer Science · Capstone Project · Academic Year 2025–2026

INSTALLATION GUIDE

Solana Local Explorer

A clean install from zero — installing Docker, building the stack, and verifying that every service came up correctly.

AUTHORS

Garvit Dadheech · Shubh Kesharwani

SUPERVISOR

Dharma Teja

TEN STEPS, ONE COMMAND

Sections 1–5 get you installed. Section 6 verifies each service independently, so if something is wrong you will know exactly which component to look at. Total time: about 15 minutes, most of it waiting for the first build.

Version 1.0 · 28 August 2026 · For day-to-day use, see the User Manual

SECTION 1

System Requirements

RESOURCE	MINIMUM	NOTES
RAM	4 GB free	Validator, PostgreSQL and two Node services run concurrently. Docker Desktop must be allowed at least this much (Settings → Resources).
Disk	4 GB free	Roughly 2 GB of images, plus the validator ledger and the PostgreSQL data volume.
CPU	2 cores	4 or more makes the first build noticeably faster.
Operating system	macOS, Linux, or Windows 10/11 with WSL2	The validator image targets `linux/amd64`. On Apple Silicon it runs under Rosetta emulation — correct, but slower to build the first time.
Network	Outbound internet	Needed once, during the first build, to pull base images and clone the Yellowstone Geyser plugin source.
Software	Docker only	You do **not** need Rust, the Solana CLI, Node.js, pnpm or PostgreSQL installed. Docker provides all of them.

SECTION 2

Install Docker

This project requires **Docker Compose V2** — the version invoked as `docker compose` (with a space), not the older hyphenated `docker-compose`. Docker Desktop 24.x and newer include it.

1 macOS

1. Download **Docker Desktop for Mac** from `docker.com/products/docker-desktop`. Choose the Apple Silicon or Intel build to match your machine.
2. Open the `.dmg`, drag Docker to Applications, and launch it.
3. Wait until the whale icon in the menu bar reports "Docker Desktop is running".

```
docker --version
docker compose version
```

> **CHECKPOINT**
> Both commands print a version. If `docker compose version` fails, you have Compose V1 only — update Docker Desktop before continuing.

2 Linux (Ubuntu / Debian)

```
curl -fsSL https://get.docker.com | sh

# run docker without sudo
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
```

On other distributions, install Docker Engine from your package manager and add the `docker-compose-plugin` package to get Compose V2.

3 Windows 10 / 11

1. Open PowerShell as Administrator and run `wsl --install`, then reboot.
2. Download **Docker Desktop for Windows** and install it, making sure *"Use WSL 2 instead of Hyper-V"* is selected.
3. Launch Docker Desktop and wait for it to report "running".
4. In Settings → Resources → WSL Integration, enable integration for your Linux distribution.

Verify from a **WSL2 terminal**, not PowerShell:

```
docker --version
docker compose version
```

WINDOWS USERS

Clone the repository *inside* the WSL2 filesystem (for example under `~/`), not under `/mnt/c/`. Bind mounts across the Windows filesystem boundary are markedly slower and can cause file-watching problems.

SECTION 3

Get the Source Code

```
git clone https://github.com/shubhiscoding/Solana-Local-Indexer-Explorer.git
cd Solana-Local-Indexer-Explorer
```

If you were given a source archive instead of repository access:

```
unzip Solana-Local-Indexer-Explorer-source.zip
cd Solana-Local-Indexer-Explorer
```

CHECKPOINT

`ls` should show `docker-compose.yml`, `validator/`, `indexer/`, `explorer/` and `README.md`. All commands in this guide are run from this directory.

SECTION 4

Check Required Ports

Four host ports are bound. If anything else already holds one, the corresponding service will fail to start.

PORT	USED BY	CHECK (MACOS / LINUX)
3000	Explorer web interface	`lsof -i :3000`
8899	Solana JSON-RPC	`lsof -i :8899`
10000	Yellowstone gRPC	`lsof -i :10000`
5433	PostgreSQL (host-mapped from 5432)	`lsof -i :5433`

On Windows PowerShell: `Get-NetTCPConnection -LocalPort 3000`, repeated per port.

If a port is taken, either stop the process holding it or remap it in `docker-compose.yml`. The left-hand number is the host side:

```
explorer:
  ports:
    - "3001:3000"     # explorer now at http://localhost:3001
```

SECTION 5

Build and Start the Stack

```
docker compose up --build -d
```

What happens, in order

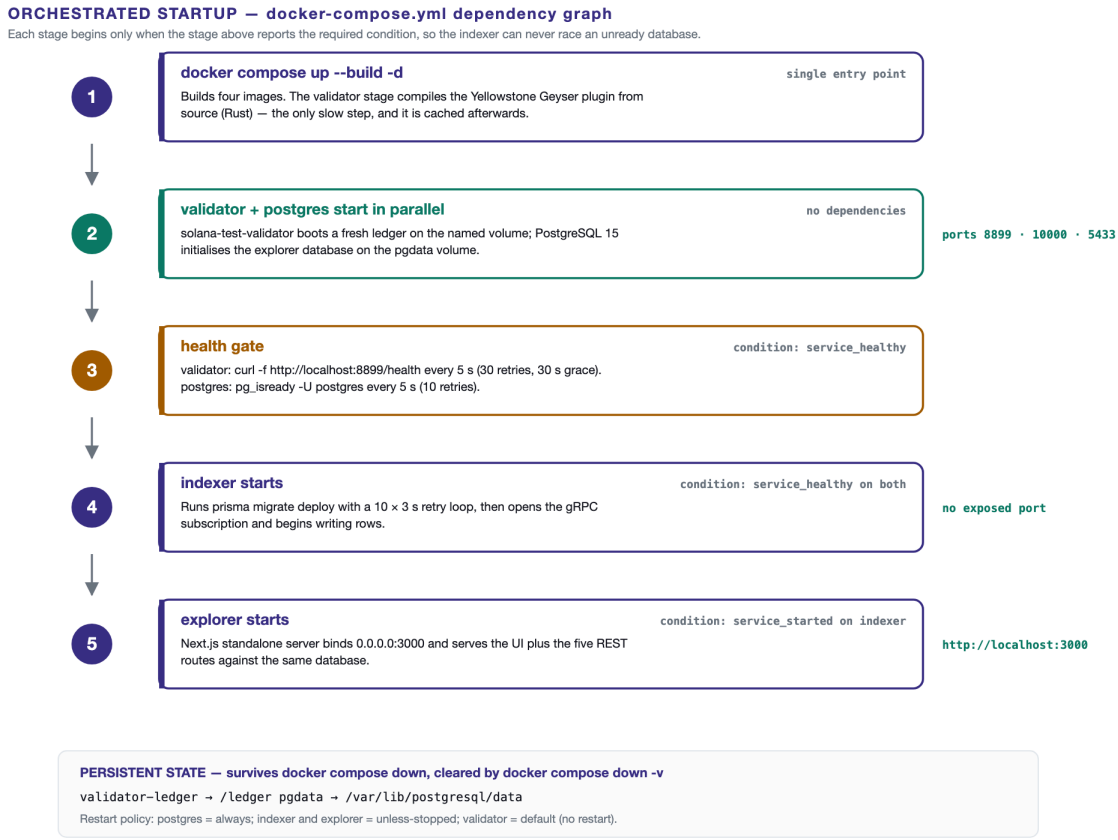


Figure 1 — The start-up sequence. Each stage waits for the previous stage to report the stated condition.

1. **The validator image builds.** This compiles the Yellowstone Geyser plugin from Rust source and is by far the slowest step. It is cached afterwards.
2. **The indexer and explorer images build.** Node/TypeScript — much quicker.
3. **validator and postgres start** and must each pass their health check.
4. **indexer starts**, applies database migrations (retrying if the database is still initialising), and subscribes to the gRPC stream.
5. **explorer starts** and serves the interface on port 3000.

> **EXPECTED FIRST-BUILD TIME**
>
> Typically **5–15 minutes**, depending on connection speed and CPU — the Rust compile dominates. On Apple Silicon, add time for emulation. Subsequent `docker compose up -d` runs (without `--build`) start in seconds. It is normal for the terminal to look idle during the compile.

SECTION 6

Verify the Installation

Five independent checks. Run them in order — each one narrows down which component to look at if something is wrong.

1 All four services are healthy

```
docker compose ps
```

```
NAME                STATUS
explorer-postgres   Up ... (healthy)
solana-explorer     Up ...
solana-indexer      Up ...
solana-validator    Up ... (healthy)
```

EXPECTED

Four rows `Up`, with validator and postgres also showing `(healthy)`. Allow up to about 60 seconds after start-up. If a service is still not healthy, go to Section 10.

2 The validator RPC responds

```
curl http://localhost:8899 -X POST -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"getSlot"}'
```

EXPECTED

A JSON response such as `{"jsonrpc":"2.0","result":1234,"id":1}`. The number will differ, and it should increase if you run the command again — that means the validator is producing blocks.

3 The explorer is reachable

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000
```

EXPECTED

`200`. Then open `http://localhost:3000` in a browser and confirm the dashboard renders.

4 The indexer is actually indexing

```
docker compose logs indexer --tail 20
```

LOOK FOR	CONFIRMS
`Migrations completed`	The schema was created successfully.
`Connecting to Yellowstone gRPC: http://validator:10000`	The endpoint was resolved.
`Subscribed to slots and transactions`	The stream is open.

LOOK FOR	CONFIRMS
`Slot 1234 status: 0`	Slot updates are arriving.
`Indexed transaction 5Kd… (slot 1234)`	Transactions are being written to the database.

5 End to end: create a transaction and watch it appear

```
docker exec solana-validator solana-keygen new \
  -o /root/.config/solana/id.json --no-bip39-passphrase --force

docker exec solana-validator solana airdrop 5 --url http://127.0.0.1:8899

docker exec solana-validator solana transfer --allow-unfunded-recipient \
  11111111111111111111111111111112 0.01 --url http://127.0.0.1:8899
```

EXPECTED

Within about five seconds, the dashboard's transaction count increases and the transfer appears at the top of the recent-transactions table. This confirms the entire pipeline — validator → Geyser → gRPC → indexer → PostgreSQL → API → interface — is working.

INSTALLATION COMPLETE

If all five checks passed, you are done. Continue with the **User Manual** for a tour of the interface and the REST API. Sections 7–9 below are optional.

SECTION 7

Optional — Local Development Setup

Only needed if you want to edit the `explorer` or `indexer` code and see changes without rebuilding an image. Requires **Node.js 20+** and **pnpm 10.x**.

```
corepack enable
corepack prepare pnpm@10.27.0 --activate
```

Explorer

```
cd explorer
pnpm install
pnpm db:generate    # regenerate the Prisma client after any schema change
pnpm dev            # http://localhost:3000, against the Dockerised database on 5433
```

Stop the containerised explorer first (`docker compose stop explorer`) so the two do not both try to bind port 3000.

Indexer

```
cd indexer
pnpm install
pnpm dev            # connects to YELLOWSTONE_ADDR, default localhost:10000
```

Stop the containerised indexer first. Running both is harmless — every write is an idempotent upsert — but it duplicates work for no benefit.

SECTION 8

Running the Test Suite

Neither suite needs a running validator, database or network.

```
cd explorer && pnpm install && pnpm test    # 9 serialisation tests
cd ../indexer && pnpm install && pnpm test   # 29 gRPC parsing tests
```

Expected output:

```
 Test Files  1 passed (1)
      Tests  9 passed (9)

 Test Files  1 passed (1)
      Tests  29 passed (29)
```

For per-case detail, add a reporter:

```
npx vitest run --reporter=verbose
```

EXPECTED

38 tests pass in total, in well under a second. The full catalogue of these plus the 54 documented functional procedures is in the *Test Cases and Validation Report*.

SECTION 9

Stopping, Resetting and Uninstalling

Stop, keeping your data

```
docker compose down
```

Containers are removed; the ledger and the database survive on named volumes. Starting again with `docker compose up -d` picks up exactly where you left off.

Reset to a clean state

```
docker compose down -v
```

Also removes the `validator-ledger` and `pgdata` volumes. The next start produces a fresh ledger and an empty, freshly migrated database.

Remove everything from your machine

```
docker compose down -v --rmi all     # containers, volumes and images
cd .. && rm -rf Solana-Local-Indexer-Explorer
```

> **IRREVERSIBLE**
>
> `-v` deletes indexed data permanently, and `--rmi all` deletes the built images, so the next build recompiles the Geyser plugin from scratch. Use plain `docker compose down` unless you genuinely want a clean slate.

SECTION 10

Installation Troubleshooting

SYMPTOM	CAUSE	FIX
`docker: command not found`	Docker not installed, or not on PATH	Reinstall per Section 2 and restart your terminal so the PATH change takes effect.
`docker compose`: unknown command	Compose V1 rather than V2	Update Docker Desktop, or install the `docker-compose-plugin` package on Linux.
Bind error / port already allocated at start-up	Another process holds 3000, 8899, 10000 or 5433	Free it, or remap the host side of the port in `docker-compose.yml` (Section 4).
The build hangs or fails during `cargo build`	Unstable network while cloning/compiling the plugin, or too little RAM	Retry `docker compose up --build -d` — Docker resumes from cached layers. Ensure Docker Desktop has at least 4 GB (Settings → Resources).
Validator never becomes `(healthy)`	Still starting, or the plugin failed to load	Wait 60 seconds (the health check allows a 30-second grace period plus 30 retries). Then `docker compose logs validator` — a dynamic-link error means a validator/plugin version mismatch.
The indexer restarts in a loop	Validator or database not healthy, or a wrong `DATABASE_URL`	`docker compose logs indexer`. Confirm with `docker compose ps` that validator and postgres are healthy *first*.
"Migration failed after all retries"	The database is genuinely unreachable	Check the postgres container is healthy and that `DATABASE_URL` points at hostname `postgres`, port 5432 — the in-network address, not the host mapping.
The explorer loads but shows zero transactions	Nothing indexed yet	Run the Section 6 step 5 commands, then check for `Indexed transaction …` in the indexer log.
The build is very slow on an M-series Mac	The validator image runs under Rosetta emulation	Expected. Only the first build is affected; later starts use the cache.
WSL2: Docker Desktop is not detected	WSL integration is off for your distribution	Docker Desktop → Settings → Resources → WSL Integration → enable it, then reopen the WSL terminal.
Prisma client errors after pulling new code	Schema changed, client not regenerated	`cd explorer && pnpm db:generate`, or rebuild with `docker compose up --build -d`.

10.1 If you need to report a problem

```
docker --version && docker compose version
docker compose ps
docker compose logs --tail 200
```

Include your operating system and architecture (for example "macOS 15, Apple Silicon"), the step in this guide where it failed, and the output above.